to ensure that the provided `id` isn't empty and that it starts with the letter `f`.

Listing 5.8 Adding basic validation to minimal API endpoints

```
using System.Collections.Concurrent;
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

var _fruit = new ConcurrentDictionary<string, Fruit>();

app.MapGet("/fruit/{id}", (string id) =>
{
    if (string.IsNullOrEmpty(id) || !id.StartsWith('f'))     ❶
    {
        return Results.ValidationProblem(new Dictionary<string, string[]>
        {
            {"id", new[] {"Invalid format. Id must start with 'f'"}}
        });
    }

    return _fruit.TryGetValue(id, out var fruit)
            ? TypedResults.Ok(fruit)
            : Results.Problem(statusCode: 404);
});

app.Run()
```

❶ Adds extra validation that the provided id has the required format

Even though this check is basic, it starts to clutter our endpoint handler, making it harder to read what the endpoint is doing. One improvement would be to move the validation code to a helper function. But you're still inevitably going to clutter your endpoint handlers with calls to methods that are tangential to the main function of your endpoint.

NOTE Chapter 7 discusses additional validation patterns in detail.

It's common to perform various cross-cutting activities for every endpoint. I've already mentioned validation; other cross-cutting activities include logging, authorization, and auditing. ASP.NET Core has built-in support for some of these features, such as authorization (chapter 24), but you're likely